



Cyberscope

A *TAC Security* Company

Audit Report

Retok Token

March 2026

Network BSC

Address 0x669db7ca691cfc63c17aDdc9fd87cb9E6A589F22

Audited by © cyberscope

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Passed
●	OTUT	Transfers User's Tokens	Acknowledged
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	MC	Missing Check	Acknowledged
●	MLI	Missing License Identifier	Acknowledged
●	MTEE	Missing Transfer Event Emission	Acknowledged
●	NWES	Nonconformity with ERC-20 Standard	Acknowledged
●	L02	State Variables could be Declared Constant	Acknowledged
●	L19	Stable Compiler Version	Acknowledged

Table of Contents

Analysis	1
Diagnostics	2
Table of Contents	3
Risk Classification	4
Review	5
Audit Updates	5
Source Files	5
Findings Breakdown	6
OTUT - Transfers User's Tokens	7
Description	7
Recommendation	7
Team Update	8
MC - Missing Check	9
Description	9
Recommendation	9
MLI - Missing License Identifier	10
Description	10
Recommendation	10
MTEE - Missing Transfer Event Emission	11
Description	11
Recommendation	11
Description	12
Recommendation	12
L02 - State Variables could be Declared Constant	13
Description	13
Recommendation	13
L19 - Stable Compiler Version	14
Description	14
Recommendation	14
Functions Analysis	15
Inheritance Graph	16
Flow Graph	17
Summary	18
Disclaimer	19
About Cyberscope	20

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	Token
Compiler Version	v0.8.7+commit.e28d00a7
Optimization	200 runs
Explorer	https://bscscan.com/address/0x669db7ca691cfc63c17addc9fd87cb9e6a589f22
Address	0x669db7ca691cfc63c17addc9fd87cb9e6a589f22
Network	BSC
Symbol	RETOK
Decimals	18
Total Supply	3,000,000,000

Audit Updates

Initial Audit	17 Mar 2026
----------------------	-------------

Source Files

Filename	SHA256
RETOK.sol	e75850037547ef5d07c0cc9053d4cdb15e4b564024f3003ef37c9f09c4414223

Findings Breakdown



● Critical	1
● Medium	0
● Minor / Informative	6

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	1	0	0
● Medium	0	0	0	0
● Minor / Informative	0	6	0	0

OTUT - Transfers User's Tokens

Criticality	Critical
Location	RETOK.sol#L30
Status	Acknowledged

Description

Any user has the authority to transfer the balance of a user's address if the user has granted allowance. The contract does not subtract the allowance in the `transferFrom()` method, as a result, the transfer can be repeated until the user's balance goes to zero.

Shell

```
function transferFrom(address from, address to, uint value)
public returns(bool) {
    require(balanceOf(from) >= value, 'balance too low');
    require(allowance[from][msg.sender] >= value,
'allowance too low');
    balances[to] += value;
    balances[from] -= value;
    emit Transfer(from, to, value);
    return true;
}
```

Recommendation

The team is advised to implement the proper allowance deduction mechanism in the `transferFrom()` method to prevent repeated unauthorized transfers. Since the contract is not upgradeable, the team is advised to migrate to a new contract with the corrected implementation.

Team Update

The team has acknowledged that this is not a security issue and states:

We are aware of a known limitation in the `transferFrom()` function: the allowance is checked but not deducted after transfer execution.

The `transferFrom()` function verifies that `allowance[from][msg.sender] >= value`, but does not subtract the value from the allowance mapping after the transfer. This means an approved spender could repeatedly call `transferFrom()` until the token holder's balance is depleted.

The contract has no proxy/upgradeable pattern and no admin functions. The deployed bytecode is immutable.

Current mitigations:

Primary trading occurs via PancakeSwap router, which only transfers exact swap amounts per transaction LP liquidity is locked via Mudra Locker. No mint/burn/pause/blacklist functions exist that could compound the risk Users are guided to approve only specific amounts, not unlimited allowances

A V2 token contract with corrected allowance deduction is planned, followed by a managed migration for all holders.

MC - Missing Check

Criticality	Minor / Informative
Location	RETOK.sol#L26,34,43
Status	Acknowledged

Description

The contract is processing variables that have not been properly sanitized and checked that they form the proper shape. These variables may produce vulnerability issues.

Specifically, the `transfer`, `transferFrom` and `approve` functions do not ensure that input addresses are not `address(0)`.

Shell

```
function transfer(address to, uint value) public  
returns(bool)  
...  
function transferFrom(address from, address to, uint  
value) public returns(bool)  
...  
  
function approve(address spender, uint value) public  
returns(bool)
```

Recommendation

The team is advised to properly check the variables according to the required specifications.

MLI - Missing License Identifier

Criticality	Minor / Informative
Location	RETOK.sol#L4
Status	Acknowledged

Description

The audited smart contract is missing an explicit SPDX license identifier, which is a crucial component for ensuring the legal clarity and compliance of the code. The license identifier specifies the terms under which the code can be used, modified, and redistributed, providing essential guidance to developers and end-users. Its absence creates ambiguity regarding the rights and obligations associated with the code, potentially leading to legal disputes or misuse.

Recommendation

The team is recommended to add an SPDX license identifier to the top of the smart contract to clearly specify the terms under which the code can be used, modified, and distributed.

MTEE - Missing Transfer Event Emission

Criticality	Minor / Informative
Location	RETOK.sol#L19
Status	Acknowledged

Description

The contract does not emit an event when portions of the main amount are transferred during the transfer process. This lack of event emission results in decreased transparency and traceability regarding the flow of tokens, and hinders the ability of decentralized applications (dApps), such as blockchain explorers, to accurately track and analyze these transactions.

```
Shell
constructor() {
    balances[msg.sender] = totalSupply;
}
```

Recommendation

It is advisable to incorporate the emission of detailed event logs following each asset transfer. These logs should encapsulate key transaction details, including the identities of the sender and receiver, and the quantity of assets transferred. Implementing this practice will enhance the reliability and transparency of transaction tracking systems, ensuring accurate data availability for ecosystem participants.

NWES - Nonconformity with ERC-20 Standard

Criticality	Minor / Informative
Location	RETOK.sol#L9,10,13
Status	Acknowledged

Description

The contract does not fully conform to the ERC20 Standard. Specifically, there are no dedicated functions for `allowance`, `totalSupply` or `decimals`. This discrepancy between the contract's implementation and the ERC20 standard may lead to inconsistencies and incompatibilities with external applications.

```
Shell
mapping(address => mapping(address => uint)) public
allowance;
uint public totalSupply = 3_000_000_000 * 10 ** 18;

uint public decimals = 18;
```

Recommendation

The incorrect implementation of the ERC20 standard could potentially lead to problems when interacting with the contract, as other contracts or applications that expect the ERC20 interface may not behave as expected. The team is advised to review and revise the implementation of the transfer mechanism to ensure full compliance with the ERC20 standard. <https://eips.ethereum.org/EIPS/eip-20>.

L02 - State Variables could be Declared Constant

Criticality	Minor / Informative
Location	RETOK.sol#L6,7,8,9
Status	Acknowledged

Description

State variables can be declared as constant using the constant keyword. This means that the value of the state variable cannot be changed after it has been set. Additionally, the constant variables decrease gas consumption of the corresponding transaction.

Shell

```
uint public totalSupply = 3000000000 * 10 ** 18;  
string public name = "RETOK Token";  
string public symbol = "RETOK";  
  
uint public decimals = 18;
```

Recommendation

Constant state variables can be useful when the contract wants to ensure that the value of a state variable cannot be changed by any function in the contract. This can be useful for storing values that are important to the contract's behavior, such as the contract's address or the maximum number of times a certain function can be called. The team is advised to add the constant keyword to state variables that never change.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	RETOK.sol#L1
Status	Acknowledged

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
Shell  
pragma solidity ^0.8.7;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

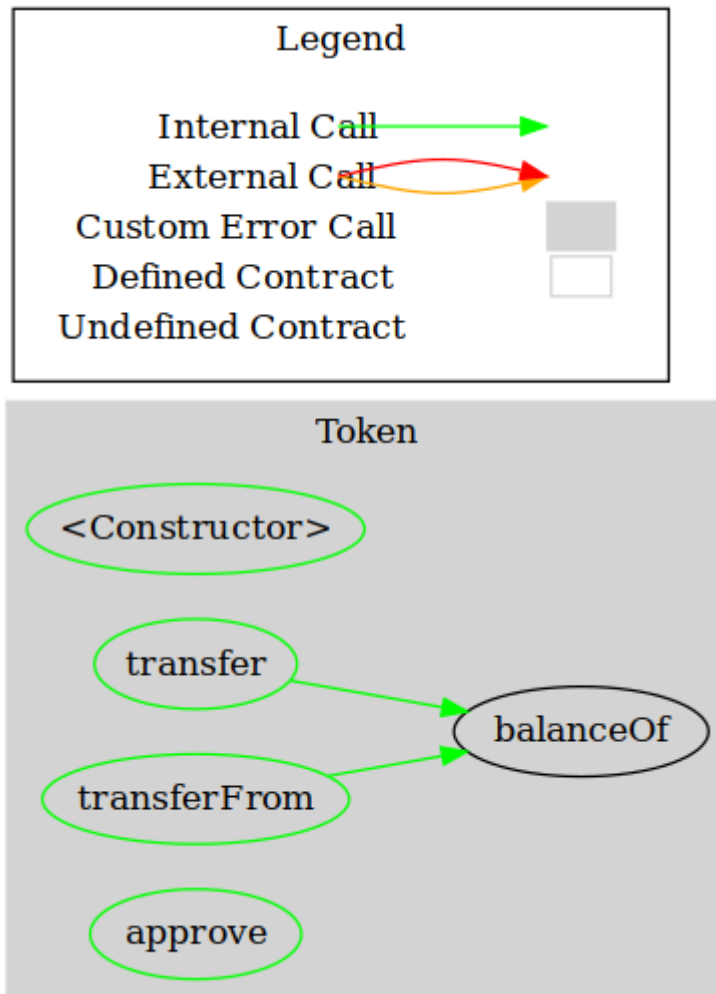
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
Token	Implementation			
		Public	✓	-
	balanceOf	Public		-
	transfer	Public	✓	-
	transferFrom	Public	✓	-
	approve	Public	✓	-

Inheritance Graph



Flow Graph



Summary

RETOK contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io